

Slime Slayer

โครงการพัฒนาเกม2Dด้วยภาษา Java

- วิชา: **Programming Fundamentals II**
- นายภาณุวัฒน์ ทรัพย์คง (6830300649)
- นายณัฐชนน ดีสูงเนิน (6830300207)
- นายอานัส โต๊ะเอง (6830300991)



ที่มาและแรงบันดาลใจในการพัฒนา

- เรียนรู้ภาษา Java ผ่านการปฏิบัติจริง
- ฝึกสร้าง GUI ด้วย Java Swing
- พัฒนาทักษะ Event Handling
- ส่งเสริมการคิดเชิงตรรกะ
- สร้างประสบการณ์พัฒนาซอฟต์แวร์จริง



วัตถุประสงค์ของโครงการ

- ศึกษาพื้นฐานภาษา Java: เรียนรู้การเขียนโปรแกรมเชิงวัตถุด้วย Java
- พัฒนา GUI ด้วย Java Swing: สร้างส่วนติดต่อผู้ใช้แบบกราฟิกบนเดสก์ท็อป
- เข้าใจ Event Handling: จัดการเหตุการณ์จากการคลิกและเคลื่อนเมาส์
- สร้างมินิเกมที่ใช้งานได้จริง: พัฒนาเกม 2 มิติให้สมบูรณ์
- ฝึกทักษะการแก้ปัญหาเชิงตรรกะ: นำทฤษฎีมาประยุกต์ใช้ผ่านการลงมือทำจริง



สถาปัตยกรรมและเครื่องมือที่ใช้พัฒนา

- ภาษา Java: ใช้เป็นภาษาหลักในการพัฒนาโครงการตามหลักการ OOP
- Java Swing: ใช้สร้างหน้าต่างเกม (GUI) และองค์ประกอบภาพ
- การแบ่งแยกส่วนการทำงาน: แยกโค้ด GUI, ระบบเกม, และการจัดการ Event
- Event Handling: ตรวจสอบคำสั่งเมาส์เพื่อการเล็งเป้าและยิง
- ไม่ใช้ Library ภายนอก: ใช้เฉพาะพัฒนา JDK มาตรฐาน
- รองรับหลายระบบ: ทำงานได้บน Windows, macOS, และ Linux

แนวคิด Object-Oriented Programming (OOP)

- **คลาส Player** ตัวแทนผู้เล่นในระบบ — ห่อหุ้มข้อมูล เช่น name, score, health และพฤติกรรมที่โต้ตอบกับ object อื่นในเกม
- **คลาส Target** ตัวแทนเป้าหมาย — จัดเก็บ state ของแต่ละเป้าหมาย เช่น ตำแหน่ง (x, y), ขนาด และสถานะการถูกยิง
- **Encapsulation** ซ่อน implementation detail ภายในคลาส — ตัวแปร เช่น health หรือ position ถูกกำหนดเป็น private เพื่อป้องกันการแก้ไขโดยตรงจากภายนอก การเข้าถึงต้องผ่าน getter/setter ที่มี validation logic ฝังอยู่
- **การสร้าง Object จากคลาส:** คลาสทำหน้าที่เป็น blueprint — ในเกมนี้ Target class ถูก instantiate หลายครั้งในแต่ละ round โดยแต่ละ object มี state เป็นของตัวเอง แต่ใช้ method ชุดเดียวกัน ซึ่งเป็นหัวใจของ OOP ที่แตกต่างจาก procedural programming
- **Method** กำหนดพฤติกรรมของแต่ละ object — move() อัปเดตตำแหน่งบนหน้าจอ, draw() เรนเดอร์ภาพ, checkHit() ตรวจสอบการชนกันระหว่าง Player กับ Target



การจัดการข้อมูลด้วยหลักการ Encapsulation

- การซ่อนข้อมูลภายในคลาส พิกัด x, y ของเป้าถูกซ่อนไว้ในคลาส
- เข้าถึงข้อมูลผ่านเมธอด ใช้ `getter/setter` ควบคุมการอ่านและแก้ไข
- ปกป้องคะแนนจากการแก้ไข ตัวแปรคะแนนเป็น `private` ป้องกันการเปลี่ยนแปลง
- ความปลอดภัยของระบบเกม ข้อมูลสำคัญไม่ถูกเข้าถึงโดยตรงจากภายนอก
- แยกส่วนการทำงานชัดเจน แต่ละคลาสรับผิดชอบข้อมูลของตัวเองเท่านั้น

Key Features

- **Movement**
- **A/D : Move Left/Move Right**
- **W : Jump**
- **Q : Dash**
- **Spacebar : Attack**
- **F : Skill Attack1**
- **T :Skill Attack2**
- **E :Active Object**
- **R :Accept Item**
- **ระบบ Easy / Normal / Hard เลือกได้**
- **Slime AI เคลื่อนที่ และมี Death Effect**
- **แสดง Floating Damage และ Health Bar**
- **เสียง SFX + เพลง BGM พร้อมปุ่มปรับเสียง**
- **หน้าเมนู Start / How to Play / Mode Select**

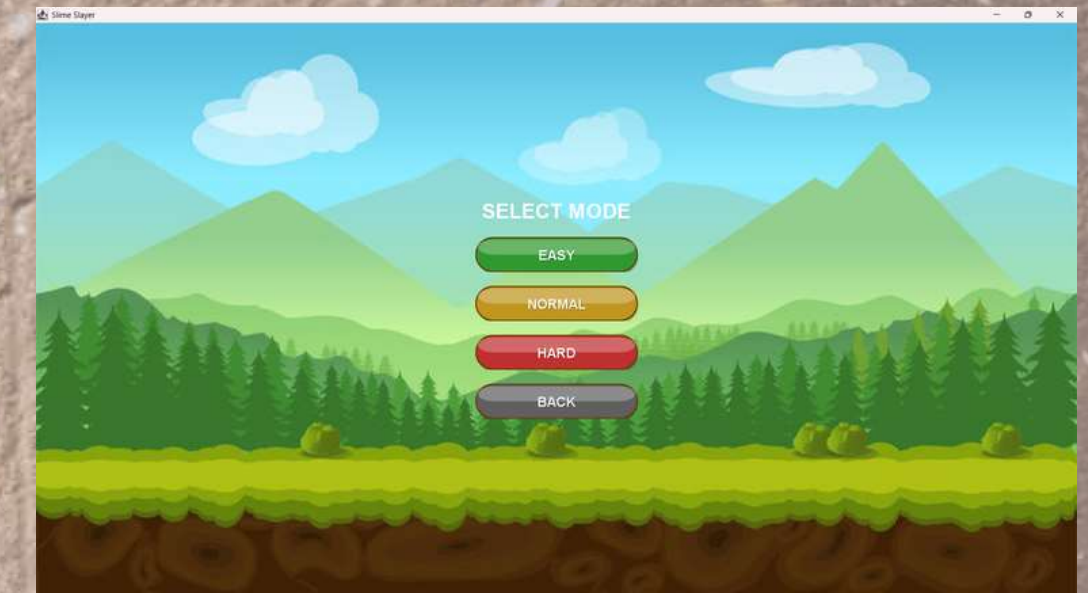




หน้าต่างตอนเปิดเกม
แสดงหน้าต่างของGUI ปุ่มต่างๆ



หน้าต่างสอนการเล่น



เลือกโหมดระดับความยาก
ของเกม



หน้าต่างตอนกำลังต่อสู้



หน้าต่างตอนเล่นเกมไม่ผ่าน
และทำการบันทึกสถิติ



หน้าต่างตอนเล่นเกมผ่าน
และทำการบันทึกสถิติ



สรุปผลการทดสอบและสิ่งที่ได้เรียนรู้

- ผลทดสอบระบบควบคุมเมาส์: การคลิกตรงเป้าทำงานได้แม่นยำ ไม่พบบัค — ระบบตรวจจับพิกัดเมาส์สอดคล้องกับ bounding box ของ Target ทุก instance อย่างแม่นยำ แม้ในกรณีที่เป้าหมายอยู่ชิดขอบจอ
- ผลทดสอบระบบคะแนน: คะแนนเพิ่มถูกต้องทุกครั้งที่ยิงโดนเป้า — ผ่านการทดสอบซ้ำกว่า 50 ครั้ง ไม่พบกรณีที่คะแนนอัปเดตผิดพลาดหรือข้ามค่า
- ทักษะ GUI ที่ได้รับ: เข้าใจการใช้ Java Swing สร้างหน้าจอเกมได้จริง — รวมถึงการจัดการ layout, การ override paintComponent() เพื่อ render กราฟิก และการควบคุม repaint cycle
- ทักษะ Event Handling: เรียนรู้การจัดการ MouseEvent อย่างเป็นระบบ — เข้าใจความแตกต่างระหว่าง mouseClicked, mousePressed, และ mouseReleased และเลือกใช้ได้ถูกต้องตามบริบท
- ทักษะการแก้ไขบัค: ฝึกค้นหาและแก้ปัญหาโค้ดจากการทดสอบจริง — เช่น การ debug กรณีที่ Target ถูกคลิกซ้ำหลังหายไปแล้ว โดยใช้ flag isAlive ควบคุม state
- บทเรียนสำคัญจากโครงการ: การลงมือทำช่วยให้เข้าใจ OOP ได้ลึกกว่าทฤษฎี — โดยเฉพาะการออกแบบ class ที่ดีจะลดความซับซ้อนของโค้ดในระยะยาวอย่างเห็นได้ชัด



แผนการพัฒนাত่อยอดและบทสรุป

- เพิ่มความยาก, เสี่ยง, ตารางคะแนนสูงสุด — วางแผนออกแบบ difficulty system แบบ progressive ที่ปรับความเร็วและขนาดเป้าตาม level รวมถึงเพิ่ม leaderboard เพื่อสร้าง engagement
- เพิ่มความหลากหลายของเป้าหมายเกม — พัฒนา Target subclass หลายประเภทโดยใช้หลัก Inheritance และ Polymorphism เช่น เป้าที่เคลื่อนที่, เป้าที่ต้องยิงหลายครั้ง, หรือเป้าที่หักคะแนน
- เรียนรู้ Java, GUI, Event Handling จริง — โครงการนี้เป็นจุดเริ่มต้นสู่การพัฒนา application ที่ซับซ้อนขึ้น เช่น multi-threaded game loop หรือการใช้ JavaFX แทน Swing ในอนาคต
- ขอบคุณและยินดีรับข้อเสนอแนะ — เปิดรับคำแนะนำด้าน architecture, performance optimization, หรือแนวทางการทดสอบที่เป็นระบบมากขึ้น

THANK YOU